• **Track identity:** track ID, title, artist.

• **Descriptive metadata:** top-level genre, duration, language code, album release date/year.

• **Rights metadata:** original license string and a flag for missing license.

• **Storage:** relative audio path, such as `000/000002.mp3`.

• Preserve missing language and release-date values as empty/null. Do not fabricate values or drop otherwise valid tracks.

• Keep the original files under `data/raw/` unchanged.

2. Development stages

Stage 1 — Build the clean music catalogue

Goal: Create a reliable table containing one row per FMA Small audio track.

• Filter `tracks.csv` using `set.subset == small`.

• Construct each expected audio path and verify that it exists.

• Select the catalogue fields listed above and retain the original track ID.

• Create `data/processed/` and write `catalogue.csv`.

• Check row count, duplicate IDs, missing required fields, and path existence.

Deliverable: `data/processed/catalogue.csv` with 8,000 validated rows.

Completion check: Exactly 8,000 unique track IDs and zero missing audio paths.

Stage 2 — Understand and inspect audio

Goal: Understand what is stored in each MP3 before extracting features.

• Load a small, representative sample of tracks in Python.

• Inspect sample rate, number of channels, duration, amplitude range and decoding errors.

• Understand waveform samples and how audio is represented digitally.

• Visualize waveforms and spectrograms for a few different genres.

• Decide a consistent audio-loading configuration, including mono/stereo handling and sample rate.

Deliverable: A tested audio-loading and inspection pipeline.

Completion check: Sample tracks load consistently and the chosen preprocessing settings are documented.

Stage 3 — Extract audio features (MFCC)

Goal: Convert each audio recording into a compact numerical representation.

• Learn the short-time Fourier transform (STFT), frequency bins and windowing.

• Understand the Mel scale, Mel filterbank and logarithmic energy.

• Understand how MFCCs are computed from log-Mel energies using a DCT.

• Extract 13 MFCC coefficient sequences per track.

• For each coefficient, calculate temporal mean and standard deviation.

Deliverable: A 26-dimensional audio feature vector for each successfully processed track.

Completion check: Feature shape, numerical validity and extraction coverage are checked across the catalogue.

Stage 4 — Prepare the recommendation feature space

Goal: Make audio and metadata features comparable and reproducible.

• Inspect distributions and outliers in the extracted MFCC statistics.

• Fit normalization/scaling parameters on the development data and save them.

• Encode genre without treating category IDs as continuous numeric values.

• Test whether genre should be used as a feature, a filter, or a separately weighted signal.

• Keep language and release year out of the initial feature space unless missingness and usefulness justify them.

• Version the feature schema and preprocessing configuration.

Deliverable: A consistent song-feature matrix plus saved preprocessing artifacts.

Completion check: The same track produces the same transformed vector when processed again.

Stage 5 — Build the first recommender: KNN

Goal: Create a simple content-based recommendation baseline.

• Implement nearest-neighbour retrieval over the song feature matrix.

• Compare suitable distance measures, such as Euclidean and cosine distance.

• Exclude the query track from its own neighbour results.

• Inspect recommendations manually across several genres and tracks.

• Evaluate similarity quality and catalogue coverage before adding adaptive ranking.

Deliverable: A working KNN recommender that returns similar catalogue tracks.

Completion check: Recommendations are reproducible, non-self matches and are inspected against baseline expectations.

Stage 6 — Build user context and feedback logging

Goal: Represent listening behaviour without prematurely assuming that every skip means dislike.

• Define events such as impression/recommendation, play, replay, completion, skip, like and explicit dislike.

• Record event timestamps and the recommendation context that produced each exposure.

• Distinguish explicit feedback from implicit behaviour.

• Define an initial reward policy, then retain raw events so reward values can be revised.

• Store user history locally rather than inside the shared catalogue.

Deliverable: A local interaction log and a documented context/reward schema.

Completion check: Every feedback event can be traced to the recommended track and the feature vector used at recommendation time.

Stage 7 — Add contextual ranking with Linear UCB

Goal: Rank KNN candidates according to the current listening context and learned user preference.

• Learn the contextual bandit setup: context, candidate/action, reward and online update.

• Implement a shared Linear UCB model over candidate feature vectors.

• Maintain per-user model state locally; initialize it from a neutral prior.

• Use an uncertainty bonus to balance exploitation and exploration.

• Update the model using the exact feature vector recorded when the recommendation was shown.

• Tune exploration strength and reward design through controlled experiments.

Deliverable: KNN candidate retrieval followed by personalized Linear UCB ranking.

Completion check: Updates are numerically stable, persist across app restarts and do not mix users' private histories.

Stage 8 — Evaluate the recommender

Goal: Determine whether adaptive ranking improves over simpler alternatives.

• Compare random recommendations, KNN only, KNN plus simple history-based ranking, and KNN plus Linear UCB.

• Use Precision@K, Recall@K, NDCG@K and HitRate@K where relevant labels exist.

• Measure catalogue coverage, skip rate, listening duration and explicit feedback.

• Test cold-start users, sparse histories, repeated listening and genre imbalance.

• Use time-aware splits where appropriate; avoid leakage from future interactions.

• Recognize that offline logs cannot reveal outcomes for items a user was never shown. Plan randomized exposure or a controlled online test for stronger evidence.

Deliverable: A reproducible evaluation report with baselines, metrics, ablations and failure analysis.

Completion check: The adaptive model demonstrates measurable improvement on appropriate metrics, or the evidence identifies why it does not.

Stage 9 — Integrate into the music app

Goal: Run recommendation inference locally and connect it to the existing player.

• Export the catalogue, feature schema, scaler and retrieval index as versioned artifacts.

• Implement local candidate retrieval and ranking in the app's supported runtime.

• Persist each user's interaction history and Linear UCB state locally.

• Download shared artifacts from a release on first setup or when an update is available.

• Test offline operation, startup time, memory use, storage footprint and recommendation latency.

• Keep user-specific state out of public releases and shared model artifacts.

Deliverable: An integrated recommendation feature that works offline against the packaged catalogue.

Completion check: The app can load artifacts, recommend tracks, record feedback and resume personalized state after restart.

3. Proposed architecture

Offline preparation on the development laptop

FMA audio + metadata → catalogue validation → audio decoding → MFCC extraction → feature normalization/encoding → song feature matrix → KNN index and versioned artifacts.

Runtime inside the music app

Listening events → user/session context → KNN candidate retrieval → candidate feature builder → Linear UCB ranking → recommended track → feedback event → local model update.

The shared catalogue and preprocessing artifacts can be distributed through GitHub Releases. Each user's interaction history and personalized bandit parameters should remain local to that user.

4. Evaluation strategy and research discipline

• **Start with a baseline:** KNN alone must work before adding Linear UCB.

• **Use ablations:** compare audio-only, audio + genre, and alternative distance/weighting choices.

• **Do not overclaim:** MFCC mean and standard deviation summarize timbral statistics; they do not represent lyrics, song structure, semantic meaning or emotion.

• **Check reward validity:** a skip can mean dislike, interruption, or context change. It is not automatically a negative preference label.

• **Control feedback loops:** a recommender can over-serve familiar genres and reduce discovery if exploration and coverage are not monitored.

• **Respect rights:** inspect the license for each recording before redistribution or commercial use. Dataset availability is not blanket permission.

5. Initial project layout

A possible structure as the project grows. It is a target organization, not a requirement to create every file immediately.

```
music-recommender/
■■■ data/
■ ■■■ raw/
■ ■ ■■■ fma_small/
■ ■ ■■■ fma_metadata/
■ ■■■ processed/
```

```
■ ■■■ catalogue.csv
■ ■■■ song_features.*
■■■ artifacts/
■ ■■■ scaler.*
■ ■■■ feature_schema.*
■ ■■■ knn_index.*
■■■ src/
■ ■■■ catalogue/
■ ■■■ audio/
■ ■■■ features/
■ ■■■ retrieval/
■ ■■■ context/
■ ■■■ bandit/
■ ■■■ evaluation/
■■■ tests/
■■■ main.py
```

Use the file format best suited to each artifact (for example, CSV for the initial catalogue; a compact binary format may be better for numerical arrays and mobile delivery).

6. Immediate next action

Do Stage 1 only. Create `data/processed/catalogue.csv` from the validated FMA Small subset. Do not extract MFCCs yet.

• Keep exactly the 8,000 FMA Small records.

• Keep missing language and release dates empty.

• Preserve the original license string, including missing values.

• Use relative audio paths so the project can be moved to another machine.

• Validate unique IDs, required fields and existence of all 8,000 audio files.

After Stage 1 passes, move to Stage 2: inspect and understand audio loading. The project should advance based on verified outputs, not merely because code runs.